

Microsoft Fabric · WMS SQL-First Medallion

PART 1 — BRONZE INGESTION

Notebook: nb_bronze_ingest_sql · 19 Delta Tables · 21 Cells · 3 Lines of PySpark

Philosophy

- All 19 tables ingested via SQL read_files() TVF — zero PySpark loops.
- Python used ONCE: 3-line schema bootstrap only.
- Each table is its own SQL cell for granular error isolation.
- delta.enableChangeDataFeed = true on raw_iot_events so Part 4 can read it incrementally.
- Run Cell 1, then Cells 2–20 in any order, then Cell 21 for validation.

CELL 1 — Schema Bootstrap (Python — only Python in this notebook)

```
for schema in ['bronze', 'silver', 'gold']:
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
print("Schemas ready: bronze | silver | gold")
```

Cells 2–20 — read_files() CTAS for all 19 Tables

Each cell follows the same pattern. Cells 2–18 use standard TBLPROPERTIES. Cell 19 (raw_iot_events) adds delta.enableChangeDataFeed = true and delta.autoOptimize.autoCompact = true for the streaming notebook in Part 4.

CELL 2 — bronze.raw_orders (50,000 rows)

```
CREATE OR REPLACE TABLE bronze.raw_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
'Files/raw/raw_orders.csv',
format => 'csv',
header => 'true',
inferSchema => 'true',
nullValue => ''
)
WHERE order_id IS NOT NULL;

SELECT 'bronze.raw_orders' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_orders;
```

CELL 3 — bronze.raw_order_lines (128,899 rows)

```
CREATE OR REPLACE TABLE bronze.raw_order_lines
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_order_lines.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE line_id IS NOT NULL;

SELECT 'bronze.raw_order_lines' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_order_lines;

```

CELL 4 — bronze.raw_customers (500 rows)

```

CREATE OR REPLACE TABLE bronze.raw_customers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_customers.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE customer_id IS NOT NULL;

SELECT 'bronze.raw_customers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_customers;

```

CELL 5 — bronze.raw_products (1,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_products
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_products.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE sku IS NOT NULL;

SELECT 'bronze.raw_products' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_products;

```

CELL 6 — bronze.raw_inventory (10,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_inventory
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,

```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_inventory.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE inventory_id IS NOT NULL;

SELECT 'bronze.raw_inventory' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_inventory;

```

CELL 7 — bronze.raw_inventory_transactions (50,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_inventory_transactions
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_inventory_transactions.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE txn_id IS NOT NULL;

SELECT 'bronze.raw_inventory_transactions' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_inventory_transactions;

```

CELL 8 — bronze.raw_shipments (30,053 rows)

```

CREATE OR REPLACE TABLE bronze.raw_shipments
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_shipments.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE shipment_id IS NOT NULL;

SELECT 'bronze.raw_shipments' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_shipments;

```

CELL 9 — bronze.raw_carriers (10 rows)

```

CREATE OR REPLACE TABLE bronze.raw_carriers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,

```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_carriers.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE carrier_id IS NOT NULL;

SELECT 'bronze.raw_carriers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_carriers;

```

CELL 10 — bronze.raw_warehouses (10 rows)

```

CREATE OR REPLACE TABLE bronze.raw_warehouses
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_warehouses.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE warehouse_id IS NOT NULL;

SELECT 'bronze.raw_warehouses' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_warehouses;

```

CELL 11 — bronze.raw_locations (2,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_locations
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_locations.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE location_id IS NOT NULL;

SELECT 'bronze.raw_locations' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_locations;

```

CELL 12 — bronze.raw_purchase_orders (8,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_purchase_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,

```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_purchase_orders.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE po_id IS NOT NULL;

SELECT 'bronze.raw_purchase_orders' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_purchase_orders;

```

CELL 13 — bronze.raw_suppliers (80 rows)

```

CREATE OR REPLACE TABLE bronze.raw_suppliers
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_suppliers.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE supplier_id IS NOT NULL;

SELECT 'bronze.raw_suppliers' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_suppliers;

```

CELL 14 — bronze.raw_supplier_performance (2,400 rows)

```

CREATE OR REPLACE TABLE bronze.raw_supplier_performance
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_supplier_performance.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE perf_id IS NOT NULL;

SELECT 'bronze.raw_supplier_performance' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_supplier_performance;

```

CELL 15 — bronze.raw_returns (5,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_returns
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,

```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_returns.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE return_id IS NOT NULL;

SELECT 'bronze.raw_returns' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_returns;

```

CELL 16 — bronze.raw_employees (200 rows)

```

CREATE OR REPLACE TABLE bronze.raw_employees
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_employees.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE employee_id IS NOT NULL;

SELECT 'bronze.raw_employees' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_employees;

```

CELL 17 — bronze.raw_labor_shifts (25,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_labor_shifts
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,
current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_labor_shifts.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE shift_id IS NOT NULL;

SELECT 'bronze.raw_labor_shifts' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_labor_shifts;

```

CELL 18 — bronze.raw_dock_activity (15,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_dock_activity
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
*,

```

```

current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_dock_activity.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE dock_id IS NOT NULL;

SELECT 'bronze.raw_dock_activity' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_dock_activity;

```

CELL 19 — bronze.raw_iot_events (100,000 rows)

```

CREATE OR REPLACE TABLE bronze.raw_iot_events
USING DELTA
TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.autoOptimize.autoCompact' = 'true',
  'delta.enableChangeDataFeed' = 'true'
)
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_iot_events.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE event_id IS NOT NULL;

SELECT 'bronze.raw_iot_events' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_iot_events;

```

CELL 20 — bronze.raw_date_dim (1,096 rows)

```

CREATE OR REPLACE TABLE bronze.raw_date_dim
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
  *,
  current_timestamp() AS _loaded_at
FROM read_files(
  'Files/raw/raw_date_dim.csv',
  format => 'csv',
  header => 'true',
  inferSchema => 'true',
  nullValue => ''
)
WHERE date_key IS NOT NULL;

SELECT 'bronze.raw_date_dim' AS table_name, COUNT(*) AS row_count
FROM bronze.raw_date_dim;

```

Cell 21 — Full Bronze Validation (UNION ALL Audit)

Row counts + PK null audit across all 19 tables in one query. All rows in the status column must read ✓ OK before proceeding to Silver (Part 2).

```
WITH audit AS (  
  SELECT 'raw_orders' AS tbl, 'order_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_orders  
  UNION ALL SELECT 'raw_order_lines' AS tbl, 'line_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN line_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_order_lines  
  UNION ALL SELECT 'raw_customers' AS tbl, 'customer_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_customers  
  UNION ALL SELECT 'raw_products' AS tbl, 'sku' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN sku IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_products  
  UNION ALL SELECT 'raw_inventory' AS tbl, 'inventory_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN inventory_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_inventory  
  UNION ALL SELECT 'raw_inventory_transactions' AS tbl, 'txn_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN txn_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_inventory_transactions  
  UNION ALL SELECT 'raw_shipments' AS tbl, 'shipment_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN shipment_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_shipments  
  UNION ALL SELECT 'raw_carriers' AS tbl, 'carrier_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN carrier_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_carriers  
  UNION ALL SELECT 'raw_warehouses' AS tbl, 'warehouse_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN warehouse_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_warehouses  
  UNION ALL SELECT 'raw_locations' AS tbl, 'location_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN location_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_locations  
  UNION ALL SELECT 'raw_purchase_orders' AS tbl, 'po_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN po_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_purchase_orders  
  UNION ALL SELECT 'raw_suppliers' AS tbl, 'supplier_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN supplier_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_suppliers  
  UNION ALL SELECT 'raw_supplier_performance' AS tbl, 'perf_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN perf_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls  
  FROM bronze.raw_supplier_performance  
  UNION ALL SELECT 'raw_returns' AS tbl, 'return_id' AS pk,  
    COUNT(*) AS rows,  
    SUM(CASE WHEN return_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
```



```

FROM bronze.raw_returns
UNION ALL SELECT 'raw_employees' AS tbl, 'employee_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN employee_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
FROM bronze.raw_employees
UNION ALL SELECT 'raw_labor_shifts' AS tbl, 'shift_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN shift_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
FROM bronze.raw_labor_shifts
UNION ALL SELECT 'raw_dock_activity' AS tbl, 'dock_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN dock_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
FROM bronze.raw_dock_activity
UNION ALL SELECT 'raw_iot_events' AS tbl, 'event_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN event_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls
FROM bronze.raw_iot_events
UNION ALL SELECT 'raw_date_dim' AS tbl, 'date_key' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN date_key IS NULL THEN 1 ELSE 0 END) AS pk_nulls
FROM bronze.raw_date_dim
)
SELECT
tbl, pk AS primary_key, rows, pk_nulls,
CASE WHEN pk_nulls = 0 THEN '✓ OK' ELSE '✗ FIX' END AS status
FROM audit
ORDER BY tbl;
-- All 19 rows must show status = '✓ OK' before running Part 2.

```

Deployment Notes

read_files() path: Use 'Files/raw/...' relative to your Lakehouse. For ADLS Gen2 replace with:
abfss://@.dfs.core.windows.net/raw/.csv

inferSchema vs. explicit schema: inferSchema works for initial builds. For production replace with an explicit SCHEMA clause to prevent silent type drift between runs.

Overwrite safety: CREATE OR REPLACE TABLE is a full overwrite each pipeline run. For incremental Bronze appends replace with INSERT INTO ... WHERE order_date > (SELECT MAX(...))

raw_iot_events — Change Data Feed: delta.enableChangeDataFeed = true is required so Part 4's streaming notebook can read this table with spark.readStream. Do NOT disable this after the stream is running.

Pipeline order: Bronze must complete fully before Silver starts. Wire in Data Factory as: [nb_bronze_ingest_sql] → success → [nb_silver_transform_sql]